

A seven-variable piecewise quadratic function without a polynomial max–min representation

Abstract

An explicit rational linear subspace of pairs of matrices identifies a continuous piecewise quadratic function on all of $\mathbb{R}^7$. The function has six polynomial labels, including zero, and admits no finite expression using maximum and minimum of real polynomials of arbitrary degrees. The proof constructs, for every finite family of quadratic tests, two points with the same test signs but different function signs. A finite exact coefficient calculation and two applications of the real implicit function theorem establish the required points. The coefficient calculation is specified and its executable verifier is included. No minimality of the dimension is asserted.

1 Statement and the explicit function

A function on $\mathbb{R}^n$ is piecewise polynomial here if a finite closed semialgebraic cover carries polynomial formulas for it. A polynomial lattice expression is a finite nonempty expression in real polynomials using pointwise minimum and maximum. Distributivity puts such an expression in finite maximum-of-minima form. Neither polynomial degrees nor the number or real coefficients of the auxiliary polynomials are restricted.

Theorem 1. *The function defined below is continuous and piecewise quadratic on all of $\mathbb{R}^7$, with six distinct polynomial labels and a six-set closed semialgebraic cover. It is not a polynomial lattice expression.*

Let $A \in \mathbb{R}^{3 \times 3}$ and $B \in \mathbb{R}^{2 \times 3}$, with no symmetry restriction on A . Number the ambient coordinates from zero, in row-major order:

$$w = (A_{00}, A_{01}, A_{02}, A_{10}, A_{11}, A_{12}, A_{20}, A_{21}, A_{22}, B_{00}, B_{01}, B_{02}, B_{10}, B_{11}, B_{12}).$$

Reorder these as

$$P = (w_0, w_3, w_6, w_1, w_4, w_7, w_{11}, w_{14}), \quad N = (w_2, w_5, w_8, w_9, w_{12}, w_{10}, w_{13}).$$

Define $T = M/630$, where

$$M = \begin{pmatrix} -210 & -210 & -210 & 420 & 210 & 0 & -210 \\ 326 & -490 & 59 & 80 & -185 & 3 & 80 \\ -276 & -210 & 66 & -270 & -150 & -228 & -270 \\ 266 & -70 & -91 & -280 & 385 & 63 & -280 \\ -210 & -210 & -210 & -210 & 210 & 0 & 420 \\ -230 & -140 & 370 & -50 & 50 & 300 & -50 \\ 36 & 0 & 279 & 90 & 225 & 153 & 90 \\ -36 & 0 & -279 & -90 & -225 & 477 & -90 \end{pmatrix}. \quad (1)$$

The vector space $W = \{P = TN\}$ is explicitly identified with $\mathbb{R}^7$ by the *unscaled* coordinates $z = N$. Thus every entry of $A(z), B(z)$ is a specified rational linear polynomial. Put

$$Q = A^\top A - B^\top B, \quad H = \text{diag}(1, 1, -1).$$

Direct rational multiplication gives $T^\top T = I_7$. Since $\text{tr}(HQ) = |P|^2 - |N|^2$, it follows that

$$Q_{00} + Q_{11} - Q_{22} = 0 \quad \text{on } W. \quad (2)$$

Set $v_i = 4e_i - \mathbf{1}$ for $0 \leq i < 3$, and $v_3 = -\mathbf{1}$, where $\mathbf{1} = (1, 1, 1)^\top$. Define

$$q_{ij} = -v_i^\top Q v_j \quad (i < j), \quad E = \{01, 02, 03, 12, 13\}, \quad h = \min_{e \in E} q_e.$$

The function of the theorem is

$$f(z) = \begin{cases} h(z), & h(z) > 0 \text{ and } \det A(z) > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

This definition applies to every $z \in \mathbb{R}^7$.

1.1 Positivity, continuity, and the closed cover

Let $u_i = (e_i + \mathbf{1})/4$ for $i < 3$ and $u_3 = 0$. The matrix $(v_i^\top Q v_j)_{i,j=0}^3$ has zero row sums, off-diagonal entries $-q_{ij}$, and $\sum_i u_i v_i^\top = I_3$. Consequently

$$Q = \sum_{i < j} q_{ij} (u_i - u_j)(u_i - u_j)^\top. \quad (4)$$

Taking H -traces and using (2) gives

$$q_{23} = q_{01} + 2q_{03} + 2q_{13}. \quad (5)$$

If $h > 0$, all six coefficients in (4) are positive. The vectors u_0, u_1, u_2 span $\mathbb{R}^3$, so Q is positive definite. Then A is invertible: $Av = 0$ would imply $v^\top Q v = -|Bv|^2 \leq 0$. Thus $\det A$ has locally constant sign wherever $h > 0$. The function is continuous there and on $\{h < 0\}$; at $h = 0$ continuity follows from $0 \leq f \leq \max(h, 0)$. Also

$$f(rz) = r^2 f(z) \quad (r > 0). \quad (6)$$

For the asserted cover, take

$$F_0 = \{\det A \leq 0\} \cup \bigcup_{e \in E} \{q_e \leq 0\}$$

with label zero and, for each $e \in E$, take

$$F_e = \{\det A \geq 0, \quad q_b \geq 0 \ (b \in E), \quad q_e \leq q_b \ (b \in E)\}$$

with label q_e . These sets are closed and semialgebraic and cover W . If $\det A = 0$ and all five q_e are nonnegative, their minimum is zero by the preceding positivity argument. This checks the formula on every boundary overlap. Distinctness of the five nonzero labels will also follow from the rank calculation below.

2 A real curve and its finite coefficient calculation

It is convenient to specify the linear constraint map by

$$(Lw)_i = s_i(P_i - (TN)_i), \quad s = (3, 630, 105, 90, 3, 63, 70, 70).$$

Its entries are integers and $\ker L = W$. The full matrix appears in the verifier in Appendix A. The base point is

$$A_0 = \text{diag}(1, -1, 0), \quad B_0 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}. \quad (7)$$

It lies in W , and its N -coordinates are $(0, 0, 0, 1, 0, 0, -1)$.

For $k = (k_0, k_1, k_2)$ define

$$d = 1 + |k|^2, \quad R_k = (1 - |k|^2)I_3 + 2kk^\top + 2[k]_\times, \quad [k]_\times v = k \times v.$$

Direct multiplication gives $R_k^\top R_k = d^2 I_3$ and $\det R_k = d^3$. For the latter identity one can also square the determinant and use its nonvanishing and its positive sign at $k = 0$. Thus $O_k = R_k/d \in SO(3)$. For $C \in \mathbb{R}^{2 \times 2}$ set

$$V = \begin{pmatrix} 1 & 0 & t_0 \\ 0 & 1 & t_1 \end{pmatrix}, \quad \Phi_A = R_k[:, 0:2]CV, \quad \Phi_B = dCV. \quad (8)$$

Here $R_k[:, 0:2]$ denotes the first two columns. The chart is polynomial in nine parameters and satisfies $\Phi_A^\top \Phi_A = \Phi_B^\top \Phi_B$ identically. If C is invertible, both matrices have rank two, and $\Phi_A = O_k[\Phi_B; 0]$.

Fix $C_{00} = 1$, put $C_{11} = -1 + t$, and regard $u = (k_0, k_1, k_2, t_0, t_1, C_{01}, C_{10})$ as seven unknowns. Let $F(t, u)$ consist of rows 1 through 7 of $L\Phi$, with row numbering starting at zero. Then $F(0, 0) = 0$ and

$$J = D_u F(0, 0) = \begin{pmatrix} 0 & 0 & 1260 & -326 & -490 & -3 & 815 \\ 0 & -210 & 0 & 46 & -35 & 38 & 25 \\ 0 & 0 & 180 & -38 & -10 & 81 & -55 \\ 0 & 0 & 0 & 1 & -1 & 0 & -1 \\ -126 & 0 & 0 & 23 & -14 & -30 & -5 \\ 0 & 0 & 0 & 66 & 0 & -17 & -25 \\ 0 & 0 & 0 & 4 & -70 & -53 & 25 \end{pmatrix}, \quad (9)$$

with

$$D = \det J = 16383079440000 \neq 0. \quad (10)$$

The real implicit function theorem supplies a smooth $u(t)$ near zero with $u(0) = 0$ and $F(t, u(t)) = 0$.

The omitted equation must be recovered. The active seven equations imply $P = TN + \ell e_0$. On the Gram-zero chart,

$$0 = \text{tr}(HQ) = \ell(2(TN)_0 + \ell). \quad (11)$$

The second factor is 2 at (7) and remains nonzero near it. Hence $\ell = 0$. Write $b(t) = \Phi(t, u(t))$, as a curve in W or in its seven coordinates. It has Gram matrix zero and rank-two matrices for all sufficiently small t .

2.1 The specified finite calculation

Over $\mathbb{Q}$, recursively define $u_{22}(t) = \sum_{j=1}^{22} u_j t^j$ by

$$u_j = -J^{-1}[t^j] F\left(t, \sum_{a < j} u_a t^a\right). \quad (12)$$

An unknown coefficient of order j enters the order- j equation only through J . Thus $F(t, u_{22}(t))$ is divisible by t^{23} , and all coefficients of u_{22} belong to $\mathbb{Z}[1/D]$. Substitute this polynomial in (8). Let C_1 be the 23×7 matrix of the N -coordinate coefficients of orders 0 through 22, and let C_2 be the 23×28 matrix for their products $N_i N_j$, ordered lexicographically for $0 \leq i \leq j < 7$. Let G_2 be the 6×28 matrix of the Gram entries, in the order 00, 01, 02, 11, 12, 22, as quadratic polynomials in N .

The following are determinants of fixed minors, with zero-based indices, reduced modulo the prime 1009:

$$\det C_1[0:7, 0:7] = 693, \quad (13)$$

$$\det C_2[0:23, 0:23] = 349, \quad (14)$$

$$\det G_2[0:5, 0:5] = 256. \quad (15)$$

Each interval includes its left endpoint and excludes its right endpoint. Appendix A implements the complete calculation by truncated polynomial multiplication and elimination. In particular, no sampling of a numerical real curve is used. The prime does not divide D or the denominators of T . Therefore reduction is defined on all coefficients in the recursion and the three displayed rational minors. Their nonzero residues prove nonvanishing over $\mathbb{Q}$ and over $\mathbb{R}$.

For clarity, the finite polynomial approximation determines the required coefficients of the *actual real curve*, without presupposing convergence of a formal power series. Set $\tilde{F} = J^{-1}F$. In a sufficiently small convex neighborhood, $\|D_u \tilde{F} - I\| < 1/2$. Integration along the segment from $u(t)$ to $u_{22}(t)$ gives

$$\|u_{22}(t) - u(t)\| \leq 2\|\tilde{F}(t, u_{22}(t))\| = O(|t|^{23}). \quad (16)$$

Both endpoints tend to zero, so the segment lies in this neighborhood for small t . Polynomial substitution in the chart, or into any fixed quadratic test, preserves this error order.

Let $\mathcal{I}_2$ denote the real span of the six Gram quadratics on W . Equation (2) and (15) give $\dim \mathcal{I}_2 = 5$. Every member vanishes on $b(t)$, hence belongs to $\ker C_2$ by (16). Equation (14) and $28 - 5 = 23$ give

$$\ker C_1 = 0, \quad \ker C_2 = \mathcal{I}_2. \quad (17)$$

The six simplex labels are an invertible linear change of the six entries of a symmetric matrix, by (4). Their unique restricted relation is (5), whose coefficient of q_{23} is nonzero. Thus the five labels used in f are linearly independent.

3 Avoiding any finite family of quadratic tests

Fix an arbitrary finite family $\mathcal{T}$ of real polynomials on W of degree at most two. For $p = c + \lambda + q \notin \mathcal{I}_2$, with homogeneous components of degrees zero, one, and two, the order- j coefficient of $p(\rho b(t))$ through order 22 is

$$\delta_{j0}c + \rho[\lambda(b(t))]_j + \rho^2[q(b(t))]_j. \quad (18)$$

At least one of these is a nonzero polynomial in ρ . Otherwise comparison of its degrees, followed by (17), would give $c = 0$, $\lambda = 0$, and $q \in \mathcal{I}_2$. For each of the finitely many $p \notin \mathcal{I}_2$, choose one such nonzero coefficient polynomial. Choose $\rho > 0$ close to 1 outside their finite sets of roots. For this fixed ρ , (16) and the first nonzero coefficient show that every one of these tests is nonzero at $\rho b(t)$ for all sufficiently small positive t . Choose a common such t and put

$$b = \rho b(t). \quad (19)$$

Both choices may be made in any prescribed neighborhoods of 1 and 0. Thus the local rank and derivative conditions needed below hold at this base. The tests in $\mathcal{I}_2$ vanish there; all remaining tests do not.

4 Two paths with the same exact Gram matrix

Fix the matrices

$$R = \begin{pmatrix} 1 & 1/4 & 0 \\ 1/4 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 1/4 & 0 \\ 0 & \sqrt{15}/4 & 0 \\ 0 & 0 & \sqrt{2} \end{pmatrix}. \quad (20)$$

They satisfy $S^\top S = R$, $\det S > 0$, $R \succ 0$ and $\text{tr}(HR) = 0$. The six simplex labels of R , in order 01, 02, 03, 12, 13, 23, are

$$(3/2, 17/2, 1/2, 17/2, 1/2, 7/2). \quad (21)$$

Keep the chosen t, ρ fixed, but temporarily allow u to vary near $u(t)$. Set $B = \rho\Phi_B$ and $O = O_k$, so the moving base matrix $A = \rho\Phi_A$ is $O[B; 0]$, irrespective of whether the linear constraints hold. Define

$$B' = BS^{-1}, \quad G = B'(B')^\top, \quad n = \frac{\text{row}_0(B') \times \text{row}_1(B')}{|\text{row}_0(B') \times \text{row}_1(B')|}.$$

Nearby, B' has row rank two, $G \succ 0$, and these functions are smooth. For small signed ε , put $M_\varepsilon = I_2 - \varepsilon^2 G^{-1}$ and

$$U = \frac{M_\varepsilon + \sqrt{\det M_\varepsilon} I_2}{\sqrt{\text{tr } M_\varepsilon + 2\sqrt{\det M_\varepsilon}}}. \quad (22)$$

All radicands are strictly positive nearby. The two-by-two Cayley–Hamilton identity gives $U^2 = M_\varepsilon$; U is symmetric and equals I_2 at $\varepsilon = 0$. Define

$$\hat{A}_\varepsilon = O \begin{pmatrix} B' \\ \varepsilon n^\top \end{pmatrix} S, \quad \hat{B}_\varepsilon = UB'S. \quad (23)$$

At zero these equal the moving scaled base chart.

The orthogonal projection onto the row space of B' is both $(B')^\top G^{-1} B'$ and $I_3 - nn^\top$. Therefore, for all nearby parameters, not merely those satisfying the constraints,

$$\hat{A}_\varepsilon^\top \hat{A}_\varepsilon - \hat{B}_\varepsilon^\top \hat{B}_\varepsilon = \varepsilon^2 S^\top (nn^\top + (B')^\top G^{-1} B') S = \varepsilon^2 R, \quad (24)$$

$$\det \hat{A}_\varepsilon = \varepsilon |\text{row}_0(B') \times \text{row}_1(B')| \det S. \quad (25)$$

Impose rows 1 through 7 of L on (23) and solve for u as a function of ε . At zero the equation holds at $u(t)$; its u -derivative is $\rho D_u F(t, u(t))$. It is invertible for sufficiently small t by (10) and continuity. The real implicit function theorem yields a smooth solution for both signs of small ε . By (24), the perturbed H -trace is zero. The active equations again give $P = TN + \ell e_0$, so (11) still holds. Its second factor is close to $2\rho > 0$; the omitted row therefore holds as well.

We have obtained a path $x(\varepsilon)$ in the *same fixed space* W , with $x(0) = b$ and

$$Q(x(\varepsilon)) = \varepsilon^2 R, \quad \text{sgn } \det A(x(\varepsilon)) = \text{sgn } \varepsilon \quad (\varepsilon \neq 0). \quad (26)$$

In particular, for sufficiently small $\varepsilon > 0$,

$$f(x(\varepsilon)) = \varepsilon^2/2 > 0, \quad f(x(-\varepsilon)) = 0. \quad (27)$$

For every $p \in \mathcal{T} \setminus \mathcal{I}_2$, $p(b) \neq 0$ and continuity gives the same nonzero sign at both points. For every $p \in \mathcal{T} \cap \mathcal{I}_2$, the exact Gram identity gives equal values at the two points, including when that common value is zero. Finiteness permits one positive ε working for all tests. We have proved:

Proposition 2. *For every finite family of real polynomials of degree at most two on W , there are two points at which all members of the family have identical signs, but f has different signs.*

The order of choices was: fixed W, f ; arbitrary finite family; ρ ; a positive t and its base; corrected paths; a positive ε . Neither W nor f depends on the tests.

5 Removing every degree bound on a representation

Lemma 3. *If a positively homogeneous function of degree two is a polynomial lattice expression, its sign is determined by the signs of finitely many real polynomials of degree at most two.*

Proof. Write $g = \mathcal{L}(p_1, \dots, p_m)$ for a finite min-max expression, and decompose each polynomial as $p_i = c_i + \lambda_i + q_i$ terms of degree at least three. For fixed x , as $r \rightarrow 0^+$, the limit of $r^{-2}p_i(rx)$ in the extended real line is

$$\begin{cases} \operatorname{sgn}(c_i) \infty, & c_i \neq 0, \\ \operatorname{sgn}(\lambda_i(x)) \infty, & c_i = 0, \lambda_i(x) \neq 0, \\ q_i(x), & c_i = 0, \lambda_i(x) = 0. \end{cases}$$

Positive scaling commutes with minimum and maximum. Homogeneity gives

$$g(x) = \mathcal{L}(r^{-2}p_1(rx), \dots, r^{-2}p_m(rx)) \quad (r > 0).$$

The two lattice operations are continuous on the extended real line, so this equality passes to their finitely many limits. After this passage, apply the sign map to the result: the sign map commutes with minimum and maximum in the ordered sets $[-\infty, \infty]$ and $\{-1, 0, 1\}$. The sign of $g(x)$ is therefore determined by the signs of the finite family $\{\lambda_i, q_i : 1 \leq i \leq m\}$ and the fixed constants c_i . No continuity of the sign map is asserted or used. $\square$

Applying Lemma 3 to (6) contradicts Proposition 2. This proves Theorem 1, including the absence of any bound on the number, real coefficients, or degrees of the auxiliary polynomials. The argument proves a finite quadratic *sign* condition; it does not assert that a lattice representation could be replaced by one whose leaves all have degree at most two.

A Executable finite certificate

The following Python 3 program uses NumPy and the standard library. It performs no random generation, numerical rank estimates, or file writes. The isometry check uses exact fractions. The Jacobian determinant uses integer Bareiss elimination. All truncated polynomial operations and subsequent elimination are modulo 1009; each multiplication is reduced before another multiplication. At order 22, the displayed dimensions and bounded residues keep all NumPy integer intermediates below 2^{63} . The first derivatives of the chart at its base have entries in $\{0, \pm 1, \pm 2\}$, so the centered modular recovery used for that integer Jacobian is exact. The characteristic-zero lifting and the comparison with the real curve are proved in the body of the paper.

The expected rank and determinant triples are (7, 693), (23, 349), and (5, 256). The output also gives the selected row and column indices; they are respectively the three fixed minors (13)–(15). A failed check raises an exception, including when Python is invoked with optimization enabled.

```

1  """Finite exact certificate for a seven-dimensional nonsymmetric Gram graph.
2
3  No random generation, numerical rank tolerance, file writes, or assertions
4  disabled by Python optimization are used. Fraction arithmetic checks the
5  graph identity; bounded integer arithmetic computes rational-jet reductions.
6  """
7
8  from fractions import Fraction
9  from itertools import combinations_with_replacement
10
11 import numpy as np
12
13
```

```

14 PRIME = 1009
15 ORDER = 22
16 POSITIVE = [0, 3, 6, 1, 4, 7, 11, 14]
17 NEGATIVE = [2, 5, 8, 9, 12, 10, 13]
18 BASE = np.array([0, 0, 0, 0, 0, 1, 0, 0, -1], dtype=np.int64)
19 UNKNOWN = [0, 1, 2, 3, 4, 6, 7]
20 PAIRS3 = list(combinations_with_replacement(range(3), 2))
21 PAIRS7 = list(combinations_with_replacement(range(7), 2))
22 L = np.array([
23     [3, 0, 1, 0, 0, 1, 0, 0, 1, -2, 0, 0, -1, 1, 0],
24     [0, 0, -326, 630, 0, 490, 0, 0, -59, -80, -3, 0, 185, -80, 0],
25     [0, 0, 46, 0, 0, 35, 105, 0, -11, 45, 38, 0, 25, 45, 0],
26     [0, 90, -38, 0, 0, 10, 0, 0, 13, 40, -9, 0, -55, 40, 0],
27     [0, 0, 1, 0, 3, 1, 0, 0, 1, 1, 0, 0, -1, -2, 0],
28     [0, 0, 23, 0, 0, 14, 0, 63, -37, 5, -30, 0, -5, 5, 0],
29     [0, 0, -4, 0, 0, 0, 0, 0, -31, -10, -17, 70, -25, -10, 0],
30     [0, 0, 4, 0, 0, 0, 0, 0, 31, 10, -53, 0, 25, 10, 70]], dtype=np.int64)
31
32
33 def require(condition, message):
34     if not condition:
35         raise RuntimeError(message)
36
37
38 def rank_minor(matrix):
39     a = np.asarray(matrix, dtype=np.int64).copy() % PRIME
40     original = a.copy()
41     row_ids = list(range(a.shape[0]))
42     columns = []
43     rank = 0
44     for j in range(a.shape[1]):
45         choices = np.flatnonzero(a[rank:, j])
46         if not len(choices):
47             continue
48         i = rank+int(choices[0])
49         a[[rank, i]] = a[[i, rank]]
50         row_ids[rank], row_ids[i] = row_ids[i], row_ids[rank]
51         a[rank] = a[rank]*pow(int(a[rank, j]), -1, PRIME) % PRIME
52         a[rank+1:] = (a[rank+1:]-a[rank+1:, j, None]*a[rank]) % PRIME
53         columns.append(j)
54         rank += 1
55         if rank == a.shape[0]:
56             break
57     rows = sorted(row_ids[:rank])
58     minor = original[np.ix_(rows, columns)].copy()
59     determinant = 1
60     for j in range(rank):
61         i = j+int(np.flatnonzero(minor[j:, j])[0])
62         if i != j:
63             minor[[i, j]] = minor[[j, i]]
64             determinant = -determinant
65         pivot = int(minor[j, j])
66         determinant = determinant*pivot % PRIME
67         minor[j] = minor[j]*pow(pivot, -1, PRIME) % PRIME
68         minor[j+1:] = (minor[j+1:]-minor[j+1:, j, None]*minor[j]) % PRIME
69     return rank, rows, columns, determinant % PRIME
70
71
72 def inverse(matrix):
73     n = len(matrix)
74     a = np.column_stack((matrix, np.eye(n, dtype=np.int64))) % PRIME
75     for j in range(n):
76         choices = np.flatnonzero(a[j:, j])
77         require(len(choices), 'singular modular inverse')
78         i = j+int(choices[0])
79         a[[i, j]] = a[[j, i]]
80         a[j] = a[j]*pow(int(a[j, j]), -1, PRIME) % PRIME
81         for i in range(n):
82             if i != j:
83                 a[i] = (a[i]-a[i, j]*a[j]) % PRIME

```

```

84     return a[:, n:]
85
86
87 def integer_determinant(matrix):
88     a = [[int(x) for x in row] for row in matrix]
89     sign, previous = 1, 1
90     for k in range(len(a)-1):
91         i = next((i for i in range(k, len(a)) if a[i][k]), None)
92         if i is None:
93             return 0
94         if i != k:
95             a[k], a[i] = a[i], a[k]
96             sign = -sign
97         pivot = a[k][k]
98         for i in range(k+1, len(a)):
99             for j in range(k+1, len(a)):
100                 numerator = a[i][j]*pivot-a[i][k]*a[k][j]
101                 require(numerator % previous == 0, 'exact Bareiss division')
102                 a[i][j] = numerator//previous
103             a[i][k] = 0
104         previous = pivot
105     return sign*a[-1][-1]
106
107
108 def chart(parameters):
109     count = parameters.shape[1]
110     zero = np.zeros(count, dtype=np.int64)
111     one = zero.copy()
112     one[0] = 1
113     mul = lambda a, b: np.convolve(a, b)[:count] % PRIME
114     k = parameters[:3]
115     norm = sum((mul(v, v) for v in k), zero.copy()) % PRIME
116     denominator = (one+norm) % PRIME
117     cross = [[zero, -k[2], k[1]], [k[2], zero, -k[0]], [-k[1], k[0], zero]]
118     nr = [(((one-norm)*int(i == j)+2*mul(k[i], k[j])+2*cross[i][j]) % PRIME
119             for j in range(3)) for i in range(3)]
120     c = [[parameters[5], parameters[6]], [parameters[7], parameters[8]]]
121     cv = [[c[i][0], c[i][1],
122             (mul(c[i][0], parameters[3])+mul(c[i][1], parameters[4])) % PRIME]
123            for i in range(2)]
124     a = [[sum((mul(nr[i][r], cv[r][j]) for r in range(2)), zero.copy()) % PRIME
125            for j in range(3)] for i in range(3)]
126     b = [[mul(denominator, cv[i][j]) for j in range(3)] for i in range(2)]
127     return np.array([a[i][j] for i in range(3) for j in range(3)]+
128                     [b[i][j] for i in range(2) for j in range(3)])
129
130
131 def gram_coefficients(basis):
132     a, b = basis[:9].reshape(3, 3, 7), basis[9:].reshape(2, 3, 7)
133     result = []
134     for i, j in PAIRS3:
135         matrix = sum((np.outer(a[r, i], a[r, j]) for r in range(3)), np.zeros((7, 7), dtype=np.int64))
136         matrix -= sum((np.outer(b[r, i], b[r, j]) for r in range(2)), np.zeros((7, 7), dtype=np.int64))
137         result.append([matrix[i, j] if i == j else matrix[i, j]+matrix[j, i] for i, j in PAIRS7])
138     return np.array(result) % PRIME
139
140
141 def main():
142     require(ORDER == 22 and PRIME == 1009, 'fixed arithmetic bounds')
143     graph = [[Fraction(-int(L[i, j]), int(L[i, POSITIVE[i]])) for j in NEGATIVE] for i in range(8)]
144     for i in range(8):
145         for j in range(8):
146             require(L[i, POSITIVE[j]] == (L[i, POSITIVE[i]] if i == j else 0), 'graph pivot coordinates')
147     for i in range(7):
148         for j in range(7):
149             require(sum(graph[k][i]*graph[k][j] for k in range(8)) == int(i == j), 'rational isometric graph')
150     require(all((630*x).denominator == 1 for row in graph for x in row), 'integer basis scaling')
151     basis = np.zeros((15, 7), dtype=np.int64)
152     basis[NEGATIVE] = np.eye(7, dtype=np.int64)
153     for i in range(8):

```



```

154     for j in range(7):
155         value = graph[i][j]
156         basis[POSITIVE[i], j] = value.numerator % PRIME * pow(value.denominator % PRIME, -1, PRIME) % PRIME
157 tangent = []
158 for j in range(9):
159     series = np.zeros((9, 2), dtype=np.int64)
160     series[:, 0], series[j, 1] = BASE, 1
161     column = chart(series)[: , 1]
162     tangent.append(np.where(column > PRIME//2, column-PRIME, column))
163 tangent = np.array(tangent).T
164 exact_jacobian = L[1:] @ tangent[:, UNKNOWN]
165 determinant = integer_determinant(exact_jacobian)
166 require(determinant == 16383079440000 and determinant % PRIME != 0, 'exact implicit Jacobian determinant')
167 active = L[1:] % PRIME
168 jacobian_inverse = inverse(exact_jacobian % PRIME)
169 series = np.zeros((9, ORDER+1), dtype=np.int64)
170 series[:, 0], series[8, 1] = BASE, 1
171 for degree in range(1, ORDER+1):
172     ambient = chart(series[:, :degree+1])
173     series[UNKNOWN, degree] = -jacobian_inverse @ (active @ ambient[:, degree] % PRIME) % PRIME
174 ambient = chart(series)
175 require(not np.any(L @ ambient % PRIME), 'all eight constraints through the certified order')
176 coordinates = ambient[NEGATIVE]
177 require(not np.any((basis @ coordinates-ambient) % PRIME), 'graph coordinate reconstruction')
178 linear = rank_minor(coordinates.T)
179 quadratic_matrix = np.array([np.convolve(coordinates[i], coordinates[j]):ORDER+1] % PRIME
180                             for i, j in PAIRS7]).T
181 quadratic = rank_minor(quadratic_matrix)
182 grams = gram_coefficients(basis)
183 gram_rank = rank_minor(grams)
184 require(not np.any((grams[0]+grams[3]-grams[5]) % PRIME), 'traceless Gram relation')
185 require(not np.any(quadratic_matrix @ grams.T % PRIME), 'Gram annihilation')
186 require((linear[0], quadratic[0], gram_rank[0]) == (7, 23, 5), 'three certified ranks')
187 require((linear[3], quadratic[3], gram_rank[3]) == (693, 349, 256), 'three nonzero minors')
188 target = [[Fraction(1), Fraction(1, 4), Fraction(0)],
189           [Fraction(1, 4), Fraction(1), Fraction(0)],
190           [Fraction(0), Fraction(0), Fraction(2)]]
191 simplex = [[4*int(i == j)-1 for j in range(3)] for i in range(3)]+[[ -1, -1, -1]]
192 labels = [-sum(simplex[i][a]*target[a][b]*simplex[j][b] for a in range(3) for b in range(3))
193           for i in range(4) for j in range(i+1, 4)]
194 require(labels == [Fraction(3, 2), Fraction(17, 2), Fraction(1, 2),
195                   Fraction(17, 2), Fraction(1, 2), Fraction(7, 2)], 'positive common target labels')
196 require(target[0][0]+target[1][1]-target[2][2] == 0, 'target preserves omitted graph constraint')
197 print('exact graph identity T^T T = I7 and integer scaling 630: passed')
198 print('exact implicit Jacobian determinant:', determinant)
199 print('linear:', linear)
200 print('quadratic:', quadratic)
201 print('Gram:', gram_rank)
202 print('common target labels:', [str(x) for x in labels])
203 print('all finite certificate checks passed')
204
205
206 if __name__ == '__main__':
207     main()

```